

```

}
pid$target->entry
/self->start/
{
  trace(timestamp);
}

pid$target->return
/self->start/
{
  trace(timestamp);
}

pid$target->"mysql_parse".return
/self->start/
{
  self->start = 0;
}

```

You will observe an output similar to the following:

```

CPU FUNCTION
0 -> __1cLmysql_parse6FnDTHD_pkclp3_v_ 1791629230
0 -> __1cLlex_start6FnDTHD_v_ 1791654796
0 -> __1cSst_select_lex_unitKinit_query6M_v_ 1791682536
0 -> __1cSst_select_lex_nodeKinit_query6M_v_ 1791710909
0 -> __1cSst_select_lex_nodeKinit_query6M_v_ 1791731630
0 -> __1cSst_select_lex_unitKinit_query6M_v_ 1791751932
0 -> __1cSst_select_lex_nodeLinit_select6M_v_ 1791776112
0 -> __1cSst_select_lex_nodeLinit_select6M_v_ 1791796307
0 -> __1cNst_select_lexKinit_query6M_v_ 1791821839
0 -> __1cSst_select_lex_nodeKinit_query6M_v_ 1791850872
0 -> __1cSst_select_lex_nodeKinit_query6M_v_ 1791871148
0 -> __1cSql_alloc6F1_pv_ 1791900080
0 -> pthread_getspecific 1791921125
.
.
.
0 -> __1cItemHcleanup6M_v_ 27789538249088
0 -> __1cItem_identHcleanup6M_v_ 27789538281270
0 -> __1cItem_fieldHcleanup6M_v_ 27789538313437
0 -> __SLIPDELETER_Q 27789538347654
0 -> __SLIPDELETER_Q 27789538382988
0 -> __1cLQdQuery_arenaKfree_items6M_v_ 27789538415179
0 -> __1cDTHDcleanup_after_query6M_v_ 27789538450969
0 -> __1cKYacc_state2T5B6M_v_ 27789538486642
0 -> __1cKYacc_state2T5B6M_v_ 27789538521089
0 -> __1cQLex_input_stream2T5B6M_v_ 27789538556597
0 -> __1cQLex_input_stream2T5B6M_v_ 27789538600602
0 -> __1cLmysql_parse6FnDTHD_pkclp3_v_ 27789538637701

```

To dig into MySQL internals, as above, please refer to forge.mysql.com/wiki/MySQL_Internals. You are also advised to refer to the book *Understanding MySQL*.

Internals by Sasha Pachev.

• **Logging queries:** So, we have watched queries go by; now how about capturing them into a file so as to use them for our own file logging purposes? We shall use a DTrace destructive function *freopen* that redirects all that is written to a standard output into the specified file. We are going to snoop on the “*dispatch_command*”: (in *sql_parse.cc*) function:

```

#!/usr/sbin/dtrace -qw
#pragma D option strsize=1024
dtrace::BEGIN
{
  freopen("/tmp/sqls");
}

pid$target->"dispatch_command".entry
{
  printf("%Y-> %s\n",walltimestamp,copyinstr(arg2));
}

```

which gives logs like:

```

2009 Feb 5 08:13:43-> create table fo_bawr (i INTEGER)
2009 Feb 5 08:13:56-> create table foo_bar (is INTEGER)

```

Using the embedded static probes

The PID provider helps us get up to speed really fast when we are learning to DTrace any user-level application. It also doesn't need a specially-built application binary. However, we need to know the source code of the application really well. A basic knowledge will enable us to write D scripts, which are only as good. DTrace static probes in an application partially reduce the need to know the code, end-to-end, in order to write useful probes. The reason is that the embedded probes can be the highest level of abstraction to the important functions that are useful and likely to be monitored for performance considerations. As noted earlier, as long as the static probes are not used, no performance hit is experienced.

Static probes are being gradually integrated into MySQL. As of MySQL 6.0.9, there are around 55 static probes. The probes are defined and documented in the *sql/probes.d* file, which is a good place to look at the currently available probes and understand how to use them in your D scripts. The currently available probes are also well described in the MySQL reference manual at dev.mysql.com/doc/refman/6.0/en/dba-dtrace-mysql-ref.html.

To enable the static probes, you will have to supply an extra option to the configure script, —*enable-dtrace*. After the build is over, start *mysqld*. Now open a terminal, and type *\$pfexec dtrace -I | grep mysql*. You should see something like the following:

```

135 mysql23509      mysqld __1cQdispatch_command6FnTernum_server_
command_pnDTHD_pcl_b_command-done
136 mysql23509      mysqld __1cQdispatch_command6FnTernum_server_

```